ABYTRONE TECHNOLOGIES · ABY POS

Aby POS: Developer Guide

For engineers working on the Aby till app. Last updated 31 October 2026, at schema version 19.

This guide explains how Aby POS is built: how the code is organised, how data moves through it, the rules the code follows, and how to make common changes safely. It sits beside two other documents:

- `README.md`: a short tour of the app, from a user's point of view.
- `docs/FEATURES.md`: the product specification. Every feature has an ID (for example `SEL-04`, `MDV-01`), and comments and commits use those IDs.

Where this guide and the code disagree, the code is right. Please fix the guide in the same commit.

Contents

1. What Aby is
2. Getting started
3. Repository tour
4. Architecture at a glance
5. Start-up and the app lifecycle
6. Dependency injection: the composition root
7. Navigation, the shell and screens
8. Feature modules
9. The data layer
10. Database migrations
11. The domain layer
12. Walkthrough: one sale, end to end
13. Staff, sign-in, permissions and approvals
14. The audit log
15. Settings
16. Licensing
17. Several devices: the local hub
18. Branches
19. Backups, restore and training mode
20. Hardware: printers, scales, scanners, screens
21. Background services
22. Reports, documents and exports
23. User interface: design system, layout and theme
24. Localisation
25. Testing
26. Building and releasing
27. Coding conventions
28. Recipes: common changes step by step
29. Pitfalls and troubleshooting
30. What is not built yet
31. Glossary

1. What Aby is

Aby is a point-of-sale app for small shops in Ghana: ice cream parlours, provision shops, chop bars, pharmacies, salons, phone shops, boutiques and building-materials stores. It runs on Windows tills, Android tablets and phones, and macOS.

The product principles in `FEATURES.md §1` shape almost every technical decision. The four that matter most for code:

Principle	What it means in the code
Complete without the internet	Every feature works against the local SQLite database. There is no server dependency anywhere. Several tills share data over the shop's Wi-Fi through a *hub* (§17), with no internet.
The device is the source of truth	The local database is the record. Sync copies rows between devices; it never decides what happened.
Nothing is lost, nothing is overwritten	Money and stock are append-only ledgers. Rows are soft-deleted (`deletedAt`), never removed. Sensitive actions go into a hash-chained audit log.
Survives power cuts	Every business action is one database transaction: the sale, its lines, payments, stock movements, outbox rows and audit entry are written together or not at all. The open cart is saved as it changes.

Other principles that affect code: the owner controls every rule through settings rather than code changes; the UI uses plain words ("Short by GH₵ 7.00", never "Variance: -700"); and money is always integer pesewas.

Technology

Concern	Choice
UI framework	Flutter (developed on 3.47.x stable), Dart SDK `^3.13.5`
State, routing, DI	GetX (`get: ^4.7.3`), used in an MVC style
Database	drift over SQLite (`drift_flutter` , `sqlite3`)
Crypto	`cryptography` (AES-GCM, PBKDF2, Ed25519), `crypto` (SHA-256, HMAC)
Documents	`pdf` , `archive` (zip), a hand-written XLSX writer, `qr` , `barcode`
Hardware	ESC/POS bytes written by hand; Windows spooler and COM ports through `ffi` ; Android Bluetooth through a method channel; `mobile_scanner` for the camera
Localisation	Flutter `gen-l10n` with ARB files

2. Getting started

Prerequisites

- **Flutter stable** (3.47 or later) on your `PATH`. Run `flutter doctor` and fix what it reports for the platforms you need.
- **Windows builds:** Visual Studio with the *Desktop development with C++* workload, and Windows **Developer Mode** turned on (`start ms-settings:developers`), because Flutter symlinks plugins.
- **Android builds:** Android Studio or the Android SDK, and a device or emulator.
- **macOS builds:** Xcode. The app is sandboxed (§26).
- **Windows installer:** Inno Setup 6 (`winget install JRSoftware.InnoSetup`).

There is no iOS or web target. drift needs native SQLite here, so web is not supported.

First run

```
flutter pub get
dart run build_runner build      # generates lib/app/data/database/app_database.g.dart
flutter run -d windows           # or: flutter run -d <android-device-id> / -d macos
```

`app_database.g.dart` is committed, but regenerate it whenever you touch `tables.dart` or the `@DriftDatabase` annotation. If generation fails silently, see §29.

The localisation classes in `lib/l10n/app_localizations*.dart` come from `flutter gen-l10n`. Because `pubspec.yaml` sets `flutter: generate: true`, they are rebuilt on every `flutter run`, `flutter test` and `flutter pub get`.

Day-to-day commands

```
flutter analyze                  # lints (analysis_options.yaml); keep it at zero issues
flutter test                     # the whole suite (about 370 cases)
flutter test test/data/sale_repository_test.dart   # one file
dart run build_runner build      # after changing drift tables
dart run build_runner watch      # keeps regenerating while you edit
```

What a fresh install looks like

A new database starts **empty**: no products, no shop name. On first open it is given the built-in roles, default reasons, payment methods, expense categories and Ghana tax presets (NHIL, GETFund, VAT; tax is off by default). The first screen (`/setup`) creates the shop and the owner's account, or offers to restore a backup or join an existing hub.

To try the app with realistic data, finish setup, then turn on **training mode** (Settings) and choose *a sample shop*. That builds a full month of made-up trading in a separate database (§19). Tests can use the old "Scoops Tamale" demo menu through `AppDatabase(executor, true)` (§25).

Licensing during development

Without a licence, the app opens read-only: you can view, export and back up, but not sell (§16). For development:

- Use **training mode**, which is always fully unlocked, or

- Make yourself a licence with `tool/license.dart` (§16). You need the private signing key. If you are not the key holder, generate your own key pair and temporarily change `licensePublicKey` locally. **Never commit that change.**

Widget tests pass `requireLicense: false` to `AbyApp`.

3. Repository tour

```
aby_pos/
├── lib/
│   ├── main.dart                  runs AbyApp
│   ├── l10n/                      app_en.arb (all UI text) + generated AppLocalizations
│   └── app/
│       ├── app.dart               GetMaterialApp, themes, training band, text size, AppRestarter
│       ├── bindings/              InitialBinding: the composition root
│       ├── routes/                route names, GetPage list, AuthMiddleware
│       ├── core/                  pure rules: permissions, typed settings, business day, tax,
│       │                          units, money denominations, scale barcodes, validators, CSV,
│       │                          licence format, report periods
│       ├── domain/                business engines: pricing, promotions, settlement, loyalty,
│       │                          inventory maths, catalogue rules and CSV import/export,
│       │                          receipts (layout, ESC/POS, PDF, QR), documents (PDF brand,
│       │                          XLSX), accountant pack, business templates, sample shops
│       ├── data/
│       │   ├── database/          drift tables, AppDatabase, migrations, schema steps, seed
│       │   ├── repositories/      the only code that touches drift (one per area)
│       │   ├── sync/row_store.dart  generic row read/apply for the hub (last write wins)
│       │   └── branch_scope.dart  which rows belong to a branch
│       ├── models/                plain Dart domain classes and enums
│       ├── services/              app-wide services (auth, session, settings, backup, hub,
│       │                          printing, scale, display, licence, notifications, ...)
│       ├── modules/               one folder per screen or feature (controllers, views,
│       │                          widgets, bindings)
│       ├── ui/                    shared widgets and helpers (the design system)
│       └── theme/                 the brand kit's Flutter theme (copied as-is)
├── test/                          unit, repository, service and widget tests
│   └── generated_migrations/      drift schema snapshots for migration tests (generated)
├── drift_schemas/                 drift_schema_v1.json … v19.json (one per schema version)
├── tool/license.dart              developer-only licence signing CLI
├── installer/                     Inno Setup script (aby_pos.iss) and the EULA
├── build_windows.ps1              one-step Windows release + installer
├── docs/FEATURES.md               product specification and phase plan
├── aby-brand-kit/                 the source brand kit (logos, fonts, tokens, web/Laravel bits)
├── assets/                        brand SVGs, app icons, UI icons, sample product pictures
├── fonts/                         Figtree, JetBrains Mono, AbyExtra (cedi sign for PDFs)
├── android/ windows/ macos/ ios/  platform runners (ios is unused)
├── build.yaml                     drift builder options
├── l10n.yaml                      gen-l10n options
└── analysis_options.yaml          lints
```

Rough sizes. About 50,000 lines of hand-written Dart in `lib/`, plus about 125,000 generated lines (drift and l10n). About 8,900 lines of tests. Big files worth knowing: `tables.dart` (1,160 lines, every table), `catalog_repository.dart` (1,000), `training_seeder.dart` (990), `settings_view.dart` (910), `app_database.dart` (810), `sale_repository.dart` (750), `hub_service.dart` (710).

Ignored by git: `*.key` and `*.abylicense` (licence keys never go in the repo), `build/`, `.dart_tool/`.

4. Architecture at a glance



The layering rules, in order of importance:

1. **Only repositories touch drift.** Controllers and services call repositories. (A few infrastructure services such as `HubService`, `BackupService`, `TrainingService` and `LicenseService` work with `AppDatabase` directly, because their job is the database itself.)
2. **Writes to syncable tables go through `insertTracked` / `updateTracked` / `softDeleteTracked`**, which add an outbox row in the same transaction (§9.4).
3. **Sensitive actions write an audit entry inside the same transaction** (§14).
4. **Permission checks go through `PermissionService.authorize()`**, which runs the approval prompt when needed. Code never checks a role by name (§13).
5. **Business arithmetic lives in `domain/` and `core/`** as pure functions, so the cart panel, checkout and the saved sale all get the same numbers.
6. **Views call controllers; controllers never build widgets** (apart from the occasional dialog helper).

5. Start-up and the app lifecycle

5.1 From `main()` to the first screen

```
main()
 └─ runApp(AbyApp)
     └─ GetMaterialApp(initialBinding: InitialBinding, initialRoute: /start)
         ├─ InitialBinding.dependencies()     registers every service and repository (§6)
         └─ /start → StartBinding → StartController.onReady → _start():
              1. await DeviceService.ready         database open, migrated; device ID read
              2. SettingsService.load()            all settings into memory, then watched
              3. BranchService.load()
              4. TrainingService.load()            is the open database the training copy?
              5. LicenseService.load()             own licence file, or the hub's
              6. apply the theme setting
              7. BackupService.finishPendingRestore()   audits a restore that just happened
              8. PriceScheduler.run()              scheduled prices whose day has come
              9. if there are staff (unawaited):   daily backup · monthly accountant pack ·
                                                   notifications
             10. start display server, start hub (unawaited; never in training)
             11. StatusService.refresh()
             12. go to /sign-in (staff exist) or /setup (fresh install)
```

Steps that could be slow or fail (backups, hub, display server) never block start-up. Their failures show up in the status centre (the dot in the header), not as crashes.

5.2 Which database file is open

`AppDatabase` opens `activeDatabaseFile()` (`app_database.dart`):

File	Where	Purpose
`aby_pos.sqlite`	data folder (below)	the shop's real data
`aby_pos_training.sqlite`	same folder	the training copy
`aby_pos.training`	same folder	marker file: while it exists, the training copy is opened
`backups/aby-*.sqlite`	application **support** folder	local snapshots (the newest 14 are kept)
`aby_pos.license`	application support folder	this device's licence (outside the database on purpose)
`aby_pos.install`	application support folder	machine ID fallback on Android (§16)
`logs/aby_pos.log`	application support folder	the crash log (§26.3)

The data folder is the application **support** folder on Windows and Linux (`%APPDATA%\Abytrone Technologies\Aby` on Windows), and the application documents folder elsewhere, where it is private to the app. On Windows the documents folder is the user's own Documents, which OneDrive syncs, and syncing an open SQLite file corrupts it. Earlier versions kept the database there: on first start `moveDatabaseFiles` copies the databases (with their `-wal`, `-shm` and `-journal` files) and the training marker across, then deletes the originals. A database already in the new folder is never replaced, and if the move fails that run uses the old folder and the next run tries again. The installer never touches these files, so uninstalling or upgrading keeps the shop's data.

5.3 Restarting in place

After a restore or entering/leaving training mode, the app must reopen a different database file. `AppRestarter.restart()` (`app.dart`) deletes every GetX instance (`Get.deleteAll(force: true)`,

`Get.reset()`) and bumps a generation key on `GetMaterialApp`, so the whole tree, `InitialBinding` included, is built again from scratch.

Consequence: never cache a service in a static or a top-level variable. After a restart it would point at a closed database.

5.4 App-wide wrappers

`AbyApp.builder` wraps every route in:

- a `Listener` that reports every pointer-down to `SessionService.touch()` (auto-lock idle timer); a `HardwareKeyboard` handler does the same for key presses;
- `_TextSize`, which applies the *Large text* device setting (text scale at least 1.2);
- `_TrainingBand`, a yellow "TRAINING" band across the top while in training mode.

5.5 Session and auto-lock

`AuthService.currentUser` holds the signed-in person **in memory only**. Restarting the app signs everyone out. `SessionService` checks every 15 seconds and locks after `AppSettings.autoLockMinutes` (default 3) of inactivity. Locking keeps the cart. The sign-in screen is `/sign-in`; every other post-sign-in route has `AuthMiddleware`, which redirects there when no one is signed in.

6. Dependency injection: the composition root

`lib/app/bindings/initial_binding.dart` is the only place app-wide objects are created. Read it top to bottom once; it is the map of the app.

Registration styles

Style	Used for	Lifetime
`Get.put(x, permanent: true)`	services and repositories needed at start or by permanent services (`AppDatabase`, `DeviceService`, `AuditRepository`, `StaffRepository`, `SettingsService`, `AuthService`, `HubService`, `CartController`, …)	until `AppRestarter`
`Get.lazyPut(() ⇒ x, fenix: true)`	most repositories	created on first `Get.find`, and **recreated** if GetX disposes of it
module `Bindings` (`Get.lazyPut(Controller.new)`)	screen controllers	while the route is on the stack

Things to know

- **Repositories take their dependencies through constructors** (`AppDatabase`, `DeviceInfo`, `AuditRepository`, other repositories). They never call `Get.find`. That keeps them testable: `test/helpers.dart` wires the same graph by hand.
- **Order matters for `Get.put`.** Anything `Get.find()`'d inside a `Get.put(...)` call must already be registered. Lazy registrations resolve when first used.
- **Callbacks break cycles.** Several services take a function instead of a dependency, for example `LicenseService(training: () ⇒ …TrainingService.active)`, `ReportRepository(branch: () ⇒ BranchService.viewing)` and `PrintService(signedIn: () ⇒ auth.currentUser.value)`.
- `PermissionService.prompter` is set to `showApprovalPrompt` here, so the service has no widget imports.
- `CartController` **is permanent**: the till and checkout share one cart, and the cart outlives sign-out and lock.
- `AccountsExportService` gets its **own** `ReportRepository` without a branch callback, so the monthly pack always covers every branch, whatever branch Reports is showing.

Adding a repository

1. Write `lib/app/data/repositories/foo_repository.dart` taking `(AppDatabase db, DeviceInfo device, AuditRepository audit, …)`.
2. Register it in `InitialBinding` with `Get.lazyPut(() ⇒ FooRepository(db, device, audit), fenix: true)`.
3. Add it to `Repos` in `test/helpers.dart` if tests need it.

7. Navigation, the shell and screens

7.1 Routes

Routes are declared in `app_routes.dart` and `app_pages.dart`. There are few of them, because most of the app lives inside the shell.

Route	Screen	Argument (`Get.arguments`)
`/start`	start-up (§5.1)	
`/setup`	first run: create the shop, restore, or join a hub	
`/sign-in`	"Who's on the till?"	
`/`	**Shell**: Sell and Manage workspaces	
`/checkout`	payment sheet over the till (transparent route, fade)	
`/close-day`	Z report and cash count	
`/staff-profile`	one staff member	staff ID
`/product`	product editor	product ID, or null for new
`/stock-item`	one stock item	stock item ID
`/receive`	receive a delivery	
`/stock-count`	a stocktake	stock count ID
`/supplier`	one supplier	supplier ID
`/refund`	refund or exchange	sale ID, or nothing (refund without a receipt)
`/customer`	one customer	customer ID
`/safe`	the safe	
`/momo`	check MoMo against the statement	

Everything after `/sign-in` has `AuthMiddleware`.

7.2 The shell

`ShellController` defines `ShellSection`, an enum of sections, each with an icon, a workspace (**Sell** or **Manage**) and the permissions that show it (`anyOf`; null means everyone signed in):

Sell workspace	Manage workspace
`sell` (the till), `today`, `orders` (only when order numbers are on)	`dashboard`, `products`, `stock`, `customers`, `money`, `reports`, `staff`, `activity`, `settings`

- **Wide screens** (≥ 840 px) show a navigation rail; **phones** a bottom bar.
- Sections are built lazily: `visited` holds those opened so far, and the others stay unbuilt.
- The header (`shell_header.dart`) holds the shop name, branch badge, status dot (status centre), command bar (Ctrl+K) and user menu (switch user, lock, clock in or out).
- Sections render inside a `SectionPage` (§23). **Sections never build their own `Scaffold` or app bar.**

7.3 Command bar

`command_bar.dart` (Ctrl+K, or the search icon) searches products, customers, sales, staff, settings and actions, and respects permissions. If you add a significant action, consider adding it here.

8. Feature modules

Each folder in `lib/app/modules/` is one feature. The usual layout:

```
modules/supplier/
├── bindings/supplier_binding.dart       Get.lazyPut(SupplierController.new)
├── controllers/supplier_controller.dart GetxController: Rx state + actions
├── views/supplier_view.dart             the page (GetView<SupplierController> or Obx)
└── widgets/                             pieces of this feature's UI
```

Modules shown as shell sections (`dashboard`, `customers`, `money`, `reports`, `staff`, `activity`, `settings`, `orders`, `today`, `till`) usually have no binding. Their controllers are put by the shell or the section view when first built.

A typical controller

From `supplier_controller.dart`:

```
class SupplierController extends GetxController {
  final SupplierRepository _suppliers = Get.find();
  final PermissionService _permissions = Get.find();

  final String supplierId = Get.arguments as String;
  final supplier = Rxn<Supplier>();
  final message = RxnString();

  Future<bool> pay(String amount, SupplierPaymentMethod method, {String? reference}) async {
    final l = appL10n();
    final pesewas = parseMoney(amount);
    if (pesewas == null || pesewas <= 0) {
      message.value = l.receivePaidInvalid;
      return false;
    }
    final auth = await _permissions.authorize(Permission.payOut,
        reason: l.supplierPayReason(supplier.value!.name));
    if (auth == null) return false;              // cancelled, or nobody approved
    try {
      await _suppliers.recordPayment(supplierId, amount: pesewas, method: method,
          reference: reference, by: auth.authority);
      message.value = l.supplierPaid(formatMoney(pesewas));
      await load();
      return true;
    } on StockException catch (e) {
      message.value = e.message;
      return false;
    }
  }
}
```

The pattern to copy:

1. Parse and validate input (`parseMoney`, `parseQuantity`, `Validators`).
2. `authorize(...)` with a human reason; stop on `null`.
3. Call the repository with `by: auth.authority`, the person whose permission was used. Pass the whole `Authorization` when both actor and approver are recorded.
4. Catch the repository's typed exception (`StockException`, `SaleException`, …) and show its `message`. Repository messages are written for the person at the till.
5. Reload state.

User-facing text comes from `appL10n()` in controllers and `context.l10n` in widgets (§24).

9. The data layer

9.1 drift setup

- All tables are in `tables.dart`. Each has a `@DataClassName('FooRow')`, so generated row classes end in `Row` and do not clash with the plain models in `lib/app/models/`.
- `build.yaml` sets `store_date_time_values_as_text: false`, so `DateTime` columns are stored as **Unix seconds** (integers). Raw SQL and test fixtures must use seconds. The audit log's `atMs` is the exception: milliseconds, stored exactly because it is hashed.
- Enum columns use `textEnum<T>()`, which stores the enum **name**. Renaming an enum value is a data migration.
- `PRAGMA foreign_keys = ON` runs in `beforeOpen`.

9.2 Sync columns

Every table that syncs between devices mixes in `SyncColumns`:

```
mixin SyncColumns on Table {
  TextColumn get id ⇒ text()();              // client-generated UUID v7
  DateTimeColumn get createdAt ⇒ dateTime()();
  DateTimeColumn get updatedAt ⇒ dateTime()();
  DateTimeColumn get deletedAt ⇒ dateTime().nullable()();   // soft delete
  DateTimeColumn get syncedAt ⇒ dateTime().nullable()();    // reserved for the cloud
  TextColumn get deviceId ⇒ text()();        // which device wrote it
}
```

- **IDs** come from `newId()` (`ids.dart`), a UUID v7: time-ordered, generated on the device, so offline tills never collide.
- **Stable IDs for shared defaults.** Rows that every device creates for itself (built-in roles, reasons, payment methods, expense categories, settings, the main branch) use fixed or name-derived IDs, so sync merges them instead of duplicating them:
  - `BuiltInRole.owner.id` = `00000000-0000-7000-8000-00000000a001`, and so on;
  - `settingRowId(key, scope, deviceId)`, `reasonRowId(...)`, `payMethodRowId(...)` and `expenseCategoryRowId(...)` are UUID v5 of a fixed string;
  - `mainBranchId = 'main'`, dated 1970 so any rename wins over it.
- **Non-synced tables** (no `SyncColumns`): `outbox`, `meta`, `hub_changes`, `hub_devices`, and the photo tables (`product_photos`, `staff_photos`, `receipt_photos`, which hold image bytes and are marked `TODO(sync)` for upload as files).

9.3 The tables

64 tables, grouped by area:

Area	Tables
Catalogue	`categories`, `products`, `product_variants` (every product has at least one), `option_groups`, `option_items`, `product_option_groups`, `tax_rates`, `product_taxes`, `barcodes`, `product_photos`, `price_changes` (scheduled and historical)
Stock	`stock_items`, `recipe_items` (what a variant or option uses), `stock_movements` (**the stock ledger**), `stock_batches` (expiry), `stock_counts`, `stock_count_lines`, `stock_transfers`, `stock_transfer_lines`

Area	Tables
Buying	`suppliers`, `supplier_ledger`, `stock_receipts` (deliveries), `stock_receipt_lines`, `purchase_orders`, `purchase_order_lines`
Selling	`sales`, `sale_lines`, `sale_taxes`, `payments`, `pay_methods`, `discounts` (saved), `promotions`, `parked_orders`, `reasons`
Refunds	`refunds`, `refund_lines`, `refund_payments`
Customers	`customers`, `customer_ledger` (credit, wallet and loyalty accounts), `gift_vouchers`, `voucher_entries`
Cash	`shifts`, `cash_events` (no-sale, pay-in, pay-out, safe drop), `safe_entries`, `momo_checks`, `momo_charges`, `day_closes`, `receipt_photos`
Money	`expense_categories`, `expenses`, `recurring_expenses`, `owner_money` (drawings and capital)
Staff	`roles`, `staff`, `staff_photos`, `time_entries` (clock in and out)
System	`audit_log`, `settings`, `outbox`, `meta`, `branches`, `device_branches`, `hub_changes`, `hub_devices`

9.4 Tracked writes and the outbox

`AppDatabase` has three write helpers for syncable tables:

```
await db.insertTracked(db.customers, CustomersCompanion.insert(...));
await db.updateTracked(db.customers, row.copyWith(name: 'Esi'));
await db.softDeleteTracked(db.discounts, id);   // sets deletedAt + updatedAt
```

Each writes the change **and** an `outbox` row (`entity`, `recordId`, `operation`, `payload` JSON) in the caller's transaction. The hub sends the outbox to other devices (§17). A future cloud sync is meant to read the same outbox.

Rules:

- Always call these **inside `_db.transaction(...)`** together with the rest of the action.
- Set `updatedAt: clock.now()` on every update. Sync is last-write-wins on `updatedAt`, so a forgotten `updatedAt` means your change loses to an older one.
- Never `delete` from a syncable table. Use `softDeleteTracked`, and filter `deletedAt.isNull()` in reads.
- Seed data and migration inserts deliberately skip the outbox (raw SQL), because every device creates the same rows itself.

9.5 Ledgers instead of balances

Balances are never stored. They are sums of append-only entries:

Balance	Ledger	Notes
Stock on hand	`stock_movements.quantity_change`	per stock item, per branch; reason `sale`, `restock`, `waste`, `adjustment`, `count`, `returned`, `production`, `transferOut`, `transferIn`
What a customer owes / holds	`customer_ledger`	accounts `credit`, `wallet`, `loyalty`

Balance	Ledger	Notes
What the shop owes a supplier	`supplier_ledger`	`purchase`, `payment`, `adjustment`
Gift voucher balance	`voucher_entries`	`issued`, `spent`, `voidReturn`, `expired`
Safe	`safe_entries`	`drop`, `deposit`, `adjustment`

To correct a balance, append a correcting entry (an adjustment, a stock count) and never edit old rows. Ledgers only ever add rows, so they never conflict during sync.

9.6 Units and money

- **Money:** integer pesewas everywhere (`int`). Doubles never touch an amount. `money.dart` has `formatMoney(1234500)` → `GH₵ 12,345.00`, `parseMoney('1,000.25')` → `100025`, and `cedis(5)` → `500`. `toCedisForDisplay` exists only for brand widgets that take `num`.
- **Costs:** stored in **thousandths of a pesewa** per base unit (`unitCostMilli`), so a 1 kg bag at GH₵ 12.00 still has an exact cost per gram.
- **Tax rates:** basis points (`1500` = 15 %). See `computeTax` in `tax.dart`.
- **Stock quantities:** integer **base units**: pieces, grams or millilitres (`StockUnit`). `DisplayUnit` converts kg and L. `formatQuantity` and `parseQuantity` are in `units.dart`.
- **Weighed products:** a sale line's quantity is grams or millilitres, and the price is per kg or litre (`SellBy`).
- **Phone numbers:** stored as `+233XXXXXXXXX` (`Validators.normalisePhone`), shown as `024 123 4567`.
- **The business day** (`business_day.dart`) starts at 05:00 by default (`AppSettings.businessDayStart`), so a sale at 01:30 belongs to the evening before. Use `BusinessDay.containing(moment, startMinutes: …)` for "today", not midnight.

9.7 Repositories

There are about 35 repositories, one per area: `catalog`, `sale`, `refund`, `customer`, `stock`, `supplier`, `receiving`, `stock_count`, `purchase_order`, `transfer`, `shift`, `safe`, `momo`, `cash_event`, `day_close`, `expense`, `profit`, `report`, `staff`, `staff_stats`, `role`, `time`, `pay_method`, `discount`, `promotion`, `voucher`, `order`, `parked_order`, `receipt`, `branch`, `settings`, `audit`, `outbox`.

Conventions:

- **Typed errors.** Each throws its own exception (`SaleException`, `StockException`, `CustomerException`, …) with a plain-English `message` for the user. Validation happens before the transaction where it can; a thrown exception means nothing was written.
- **`by:` parameter.** Write methods take the `Staff` responsible, for the audit entry and `byName` columns.
- **Streams for live lists** (`watch…`), futures for one-off reads.
- **Time through `package:clock`** (`clock.now()`), never `DateTime.now()`, in repositories. The training seeder replays a month with `withClock(...)`, and tests can fix the time.
- **Branch scope.** Reads that are per branch (stock on hand, sales in reports) filter through `BranchFilter` (§18).

9.8 Meta

`meta` is a key/value table for **device-local** values that never sync: `device_id`, backup timestamps, the backup password hash, hub link details (`hub.role`, `hub.url`, `hub.secret`, `hub.cursor`, …), the hub's licence, the training marker keys and the pending-restore note. Use `db.readMeta` / `writeMeta` / `deleteMeta`.

10. Database migrations

Installed tills keep their data, so **every** schema change needs a migration. Migrations use drift's step-by-step API with generated schema snapshots.

10.1 The procedure

1. Edit `tables.dart` (and the `@DriftDatabase(tables: [...])` list if you add a table). Bump `schemaVersion` in `app_database.dart`.
2. `dart run build_runner build`
3. `dart run drift_dev schema dump lib/app/data/database/app_database.dart drift_schemas/drift_schema_vN.json`
4. `dart run drift_dev schema steps drift_schemas lib/app/data/database/schema_versions.dart`
5. Add `fromXToY: (m, schema) async { ... }` to `stepByStep(...)` in `app_database.dart`. Use the versioned `schema.*` objects, **not** the current table classes:

```
from17To18: (m, schema) async {
  await m.addColumn(schema.categories, schema.categories.icon);
},
```

6. `dart run drift_dev schema generate --data-classes --companions drift_schemas test/generated_migrations`
7. Add `N - 1` to the loop in `test/data/migration_test.dart`, change its target version to `N`, and add a data test if you move or transform data.

10.2 Writing migration steps

- New columns on existing tables need a default (`withDefault`) or must be nullable.
- Steps that insert data use **raw SQL** helpers (`_insertBuiltInRoles`, `_grantToManager`, ...), because the current table classes may not match the schema at that step.
- Do not write to the outbox in migrations. Every device runs the same migration.
- To give an existing role a new permission, follow `from18To19`: `_grantToManager(...)`.

10.3 Schema history

v	Change
1	Baseline: offline till, checkout, today
2	Staff sign-in and day close
3	Roles and permissions, audit log, settings (shop name moved from meta)
4	Staff profiles and photos
5	Catalogue and stock: variants everywhere, option groups, stock items, recipes, taxes, barcodes
6	Selling: open price and weighed products, the Quick tab, discounts, parked orders
7	Payment methods, split payments, cash rounding, refunds and exchanges
8	Customers, the credit book and wallets (old free-text credit names become customers)
9	Shifts, note and coin counts, pay in / out, the safe, MoMo checks

v	Change
10	Expenses, recurring expenses, owner drawings and capital
11	Automatic promotions, promotion amounts on sale lines
12	Gift vouchers
13	Queue numbers and prep status on sales
14	Purchase orders, batches and expiry, production
15	Badges, usual hours, the time clock, commission per product
16	The hub's change log and paired devices
17	Branches, devices in branches, staff per branch, transfers
18	A picture for each category
19	`manageDevices` permission, granted to Manager
20	`staff.staff_number`, numbered from 101 by `created_at`
21	`staff.recovery_hash`: owner recovery codes

11. The domain layer

`lib/app/core/` and `lib/app/domain/` hold the rules as pure Dart. They are the most heavily tested code (`test/domain/`).

11.1 Pricing: `CartPricing` (`pricing.dart`)

One class gives the cart panel, checkout and `SaleRepository.completeSale` the same numbers. In order:

1. Line price = variant price + option deltas (or an override or open price) × quantity.
2. Manual **line discounts**.
3. Automatic **promotions** (and loyalty rewards, which are modelled as promotions).
4. The **order discount**, a percentage or amount of what is left, shared across lines in proportion using the *largest remainder* method, so the shares add up exactly.
5. **Tax** per line on what is left (`computeTax`): `off`, `inclusive` (prices include tax; the usual case in Ghana) or `exclusive` (added at the till). Compound rates apply on base plus the simple taxes. Inclusive leftovers go to the base, so the parts always add up to the price.

11.2 Promotions (`promotions.dart`)

A `Promotion` has a schedule (date range, weekdays bitmask with bit 0 = Monday, a time window that can wrap past midnight) and a `PromoRule` (stored as JSON on the row):

Kind	Meaning
`percentOff`	percent off a target (happy hour)
`quantityBreak`	N or more of a target cost X each
`buyGet`	buy N, get M of a reward target at P % off (100 = free); the cheapest qualifying items are the reward
`bundle`	one of each part together for X
`spend`	spend ≥ threshold → percent off, amount off, or the cheapest reward item free

`PromoTarget` picks products, categories, or everything. When promotions compete, the best deal for the customer wins.

11.3 Settlement (`settlement.dart`)

`Settlement.of(total, tenders, roundTo:)` covers a total with payments in order (split payments). Only the last payment may exceed what is left, and only in cash (that gives change). When the last payment is cash and rounding is on (10 or 50 pesewas), the cash due is rounded half-up and the difference is recorded as `sales.rounding`. The payments add up to `total + rounding`.

11.4 Other engines

File	What it does
`loyalty.dart`	stamps, points or Gold tier (one style per shop); rewards become promotions
`inventory.dart`	weighted average cost, suggested low level, days of stock left
`catalogue_rules.dart`	validation for every catalogue change (controllers call this, not the repository's write methods)

File	What it does
`catalogue_io.dart`	CSV export and import with a preview; import never removes anything
`orders.dart`	queue numbers ("A23") per business day; which items need prep
`business_templates.dart`	nine shop kinds (`BusinessType`) with starting categories and feature settings
`sample_shops.dart`	the sample data training mode builds for each shop kind
`audit_details.dart`	turns audit `details` JSON into readable before/after rows
`customer_documents.dart`, `purchase_order_pdf.dart`	statements and purchase orders as PDF
`labels.dart`	price and barcode labels (A4 sheets, rolls, receipt-printer stickers)
`scale_reading.dart`	parses what serial and Bluetooth scales send
`core/scale_barcode.dart`	EAN-13 barcodes printed by shop scales (prefix, item code, weight or price)
`core/denominations.dart`	Ghana notes and coins; `CashCount` for drawer counts
`core/period.dart`	report periods and `DateRange` (with the period before, for comparison)
`core/csv.dart`	minimal RFC 4180 CSV (handles Excel's byte-order mark)

11.5 Receipts (`domain/receipt/`)

- `receipt_builder.dart` builds a device-independent document: sale receipts (customer and shop copies for credit), kitchen tickets, X and Z reports, customer balance slips, gift vouchers, staff badges and the printer test page. Printed text is English, because customers and accountants read it.
- `escpos.dart` turns that document into ESC/POS bytes for 58 mm and 80 mm thermal printers: text, QR codes and Code 128 barcodes as bitmaps, a dithered logo (at most 384 dots wide), the drawer-kick pulse and a partial cut. Images print in bands of 128 rows for printers with small buffers.
- `receipt_pdf.dart` gives the same receipt as a PDF (sent by WhatsApp, SMS or email).
- `receipt_code.dart` puts the sale ID plus a check code in the receipt's QR code. Scanning it at the till or in Refund opens the sale. Mistyped or made-up codes are refused.

12. Walkthrough: one sale, end to end

1. **Ring up** (`till_controller.dart`, `cart_controller.dart`). Taps, scans, short codes and scale barcodes add `CartLine`s. Weighed items ask `ScaleService` or the cashier for grams. The cart is serialised to JSON and saved through `ParkedOrderRepository` on every change, so a crash loses nothing. Removing lines or clearing the cart is audited.
2. **Pricing** is recomputed reactively: `cart.pricing` is a `CartPricing` with the shop's promotions (filtered to those running now), loyalty for the attached customer, the order discount and the tax mode.
3. **Checkout** (`/checkout`, a transparent route over the till). `CheckoutController` collects one or more payments, checks MoMo references and voucher codes, works out change and cash rounding, and tells the customer display what to pay (`DisplayService.paying`).
4. **Save** with `SaleRepository.completeSale(...)`. Before the transaction it validates the lines, re-prices with the same engines, and builds the `Settlement`. Then **one transaction**:
   - checks each payment reference has not been used before (a MoMo SMS shown twice is a payment taken twice; short references are checked over a time window);
   - checks the wallet balance and credit limit (over the limit needs `creditApprover`, or is blocked, depending on `credit.overLimit`);
   - writes `sales` (next per-device number, queue number if on), `sale_lines`, `sale_taxes`, `payments`;
   - writes `stock_movements` from each line's recipe (variant and option recipes; batches earliest-expiry first);
   - writes customer ledger entries (credit sale, wallet spend, change kept in the wallet, loyalty stamps or points), voucher entries, and refund rows for an exchange;
   - writes audit entries for discounts and price overrides;
   - writes outbox rows for all of the above (through `insertTracked`).
5. **After saving:** the cart resets, `PrintService` queues the receipt (and kitchen ticket, and opens the drawer for cash). The print queue retries on its own, so a printer problem never holds up a sale. The display shows "thank you" and change for 20 seconds. If the device is a hub member, the next sync tick (every 3 seconds) sends the sale.

Voids (`SaleRepository.voidSale`) need a reason and `Permission.voidSale`. They put stock back, and are refused for sales in a closed day. **Refunds** (`RefundRepository`) can restock items, pay back by any method, and run inside an **exchange**, where the returned value pays for new items in the same transaction.

13. Staff, sign-in, permissions and approvals

13.1 Staff and PINs

- Staff sign in with a 3-digit **staff number** and a 4-digit PIN. A badge QR code, or being the only person at the branch, stands in for the number. The sign-in screen, Switch user and the approval prompt never list names, and PINs need not be unique: the number identifies the person (`StaffRepository.withNumber`). Numbers are given out from 101, lowest free first. Two tills offline can hand out the same number, so `withNumber` tries every match, and `ensureNumbers()` (run on the sign-in screen) numbers rows synced from older versions. PINs are stored as `v1$<iterations>$<salt>$<hash>`: PBKDF2-style HMAC-SHA256, 20,000 iterations, 16-byte salt, compared in constant time (`pin_hasher.dart`). Four digits cannot resist someone who has the database file; the hashing slows them down, and sign-in lockout handles guessing at the till.
- **Lockout:** 5 wrong tries in a row lock that staff number for 30 seconds (a badge sign-in counts against the same number). Each lockout after that doubles, up to 15 minutes; a lockout that ended over an hour ago is forgiven, and signing in clears the count. Lockouts are kept in the `meta` table (`auth.lockouts`, this device only), so restarting the app does not reset them. A wrong number and a wrong PIN give the same message. Approvals have their own shared counter. Failures and lockouts are audited.
- Staff can be limited to one branch (`staff.branch_id`). Owners can sign in at any branch.
- **Forgotten owner PIN** (`pin_recovery_service.dart`, `core/pin_reset.dart`). Anyone with *Manage staff* can set someone else's PIN, so this is only for owners locked out of their own shop. There are two ways back in, and both end with a new PIN, a new recovery code and an audit entry (`staffPinReset`):
  - **Recovery code:** 12 characters (`K7QM-4XPA-9TRB`), hashed into `staff.recovery_hash` with `PinHasher`. Made at setup and shown once, remade from the owner's own profile (after `confirmOwnPin`), replaced each time it is used. Wrong codes share one lockout (`#recovery`). The status centre warns an owner who has none.
  - **Supplier reset:** the till makes a `PinResetRequest`: device code, staff number and an 8-character nonce. It is kept in `meta` (`pin.reset.request`, this device only, 7 days). The supplier signs `aby-pin-reset:v1:<device>:<staff>:<nonce>` with the licence key (`tool/license.dart reset`). The till rebuilds that message and checks the Ed25519 signature against `licensePublicKey`, so the code is only the signature. It works once, on that device, for an active owner. Whether a number is an owner's is checked only when the code comes back. The supplier may answer hours later, so nothing waits in the dialog. While a request is open, the sign-in link reads "Enter your supplier's reset code" and the dialog opens on it. `cancelRequest()` ("Start again") drops it. When the owner it is for signs in with their PIN (sign-in or Switch user), `dropIfRemembered` drops it too, so a late reset code does nothing. Getting in with the recovery code also drops it. Each cancellation is audited (`staffPinResetCancelled`).

13.2 Permissions and roles

`permissions.dart` defines the `Permission` enum in six groups: `sell`, `catalogue`, `customers`, `cash`, `reports`, `admin`. Roles store their permissions **by name**, so never rename a value without a migration. Features not yet built already have their permission, so roles set up today keep working.

Built-in roles (fixed IDs):

Role	Permissions	Max discount without approval
Owner	everything, always (its list is ignored)	100 %

Role	Permissions	Max discount without approval
Manager	everything except `manageRoles`, `restoreBackup`, `writeOffCredit`	50 %
Cashier	`sell`, `closeDay`	10 %
Stock keeper	`receiveStock`, `adjustStock`, `editProducts`	0 %

Owners can create custom roles (Staff → Roles).

13.3 Checking a permission

```
// Can the signed-in person do it? (for showing or hiding UI)
if (Get.find<PermissionService>().can(Permission.refund)) { ... }

// Doing it: the signed-in person, or someone who approves with their PIN.
final auth = await permissions.authorize(Permission.voidSale, reason: 'Void sale #00142');
if (auth == null) return;          // cancelled, or nobody allowed approved
await sales.voidSale(id, actor: auth.actor, approver: auth.approver, reason: reason);

// Discounts are checked against the role's limit.
final auth = await permissions.authorizeDiscount(25, reason: '...');
```

`Authorization.authority` is the person whose permission was used (the approver if there was one). Both actor and approver go into the audit log.

13.4 The licence gate

`Staff.can()` also consults `PermissionGate.allows`, a global switch that `LicenseService` narrows when there is no licence: only `reports` and `admin` permissions work, so an owner can always see, export and back up their own data (§16). Tests run with the gate open.

14. The audit log

`audit_repository.dart` keeps a tamper-evident log. Each device has its own chain:

```
hash = hex SHA-256 of these fields joined with U+001F (unit separator), nulls as '':
       'v1', seq, deviceId, atMs, action, actorId, actorName, approverId, approverName,
       entity, entityId, summary, details, prevHash
first entry: prevHash = "000…0" (64 zeros)
```

The `'v1'` prefix versions the hash recipe. If you ever add a hashed field, add a `'v2'` recipe and keep verifying old entries with `'v1'`.

- `record(action:, summary:, actor:, approver:, entity:, entityId:, details:)` appends an entry. **Call it inside the transaction of the action it records.** It opens a nested transaction, so it also works on its own.
- `details` is JSON, typically `{"before": …, "after": …}`. `audit_details.dart` renders it.
- `verifyChain()` walks every device's chain and reports the first entry whose hash, link or sequence is wrong. The status centre and Activity screen show a warning when someone has edited the database file outside Aby.
- `AuditAction` (in `models/audit.dart`) has about 90 values. Add a new one at the end; names are stored.
- Audit rows sync like any other row. Each device's chain stays intact because entries are keyed by `(deviceId, seq)`.

What gets audited: sign-in, failures, locks, approvals refused; voids, refunds, discounts, price overrides, cart removals, no-sales; all catalogue, stock, supplier, customer, cash, expense, staff, role, setting, branch, device, backup, export, licence and training actions.

15. Settings

15.1 Declaring a setting

All settings are declared in `core/settings.dart`:

```
static const autoLockMinutes = Setting<int>('security.autoLockMinutes', SettingScope.shop, 3);
static const productView = Setting<String>('device.productView', SettingScope.device, 'auto', audited:
false);
```

- **Scope.** `SettingScope.shop` settings sync to every device (one shared row). `device` settings belong to one device (printer, paper width, theme, scale).
- **Type safety.** Values are stored as JSON. A stored value of the wrong type falls back to the default (`decode`).
- **`audited: false`** for personal preferences anyone may flip, so they do not bury real changes in the activity log.
- Add every new setting to `AppSettings.all`, which drives loading, `byKey` and the labels in `ui/setting_labels.dart`.
- Enumerated settings are strings (`'off' | 'inclusive' | 'exclusive'`) converted at the point of use (`TaxMode.fromName`).

15.2 Reading and writing

```
final settings = Get.find<SettingsService>();
final minutes = settings.get(AppSettings.autoLockMinutes);       // reactive inside Obx
await settings.set(AppSettings.autoLockMinutes, 5, by: auth.authority);
```

`SettingsService` keeps everything in an `RxMap` and watches the table, so a shop setting changed on another till (arriving through the hub) updates screens by itself. The caller checks `Permission.changeSettings` first.

16. Licensing

16.1 Model

- A licence is a **one-time purchase for one shop**. It is tied to one **device code** (the hub, or a till on its own) and covers N devices in total, that device included. It never expires.
- Hub members do not need their own licence: the hub sends its licence key in every sync reply, and the hub refuses to pair more devices than the licence covers (`HubError.noSeats`).
- **Without a licence** the app works read-only (`PermissionLicensing.worksUnlicensed`): reports and admin (staff, settings, devices, backups, exports) work; selling, stock, customers and cash do not. **Training mode is always fully open**; it is the demo.

16.2 Format (`core/license.dart`)

```
ABY1.<base64url payload>.<base64url Ed25519 signature>
payload = {"v":1,"id":"<uuid>","shop":"Mama Ama Provisions","device":"K7QM-2XRA-PL3D-
9WEH","devices":3,"issued":"2026-10-31"}
```

- Whitespace is ignored, because messaging apps insert line breaks.
- The app verifies with `licensePublicKey`. **Changing that key voids every licence sold.**
- The **device code** is 16 characters from SHA-256 of `aby-device:<machine id>`, using an alphabet with no 0/O/1/I, shown in groups of four. The machine ID (`machine_id.dart`) is:
  - Windows: `HKLM\SOFTWARE\Microsoft\Cryptography\MachineGuid` (read through FFI);
  - macOS: `IOPlatformUUID` from `ioreg`;
  - Linux: `/etc/machine-id`;
  - Android and others: a UUID kept in `aby_pos.install`, so it changes when the app is reinstalled.
- The licence file lives **outside the database** (`aby_pos.license` in app support), so a restore or training never affects it. The hub's licence, as a member last heard it, is in meta (`hub.license`).

16.3 Issuing licences: `tool/license.dart` (developer only)

```
dart run tool/license.dart keygen <folder>        # once: makes the private key, prints the public key
dart run tool/license.dart issue --key <folder> --shop "Mama Ama Provisions" --device K7QM-2XRA-PL3D-
9WEH --devices 3
dart run tool/license.dart check <key or .abylicense file>
dart run tool/license.dart reset --key <folder> --request K7QM-2XRA-PL3D-9WEH-101-7QXM-P3RA
```

`issue` prints the key, saves `<Shop>-<code>.abylicense` in the current folder, and appends to `issued.csv` in the key folder. The shop pastes the key in Settings → Licence (or opens the file).

`reset` answers an owner who forgot their PIN and has no recovery code (§13.1). It prints who the device is licensed to (from `issued.csv`), the staff number and a reset code (`ABYP.<signature>`), and appends to `resets.csv`. Confirm you are talking to that shop's owner before sending it.

Security: the private key (`aby_license_private.key`) must never be committed, shared or shipped. `.gitignore` excludes `*.key` and `*.abylicense`. Back the key up offline: without it no more licences can be issued, and replacing it voids all existing ones.

17. Several devices: the local hub

`hub_service.dart` implements `MDV-01`. One device (usually the main till) is the **hub**; the others are **members**. They sync over the shop's Wi-Fi with no internet. Each device keeps working alone when the network drops, and catches up later.

17.1 Roles and ports

HTTP server on the hub	TCP **8787**, all IPv4 interfaces
Discovery beacon	UDP broadcast to **47878** every tick: `{"aby":"hub","h":<hubId>,"p":<port>}`
Tick	every **3 s** (hub: move its own outbox into the log and broadcast; member: sync)
Endpoints	`GET /hello`, `POST /pair`, `POST /sync`

The Windows installer adds a firewall rule for `aby_pos.exe`, so cashiers never see a firewall prompt. Android sets `usesCleartextTraffic="true"` because the hub speaks plain HTTP; all payloads are encrypted at the application level instead (below).

17.2 Becoming the hub

`becomeHub()` copies **every** synced row into `hub_changes` (the change log), clears the outbox, and stores `hub.role = hub`, `hub.id` and a new log **epoch**.

17.3 Pairing

1. On the hub, *Add a device* → `offerPairing()`: an 8-character one-time code (no 0/O/1/I/L), valid for 10 minutes, shown as a QR code `ABY-HUB:{"u":[urls],"c":code,"h":hubId}`.
2. The member scans or types it. Both sides derive a key from the code with PBKDF2-HMAC-SHA256 (20,000 iterations, salt `aby-hub:<hubId>`), slow on purpose so a recorded pairing cannot be brute-forced.
3. The member sends a hello (device ID, name, version) sealed with that key. The hub allows 5 tries per code, checks seats against the licence, makes a fresh **32-byte device secret**, stores it in `hub_devices`, and returns it sealed.
4. Everything afterwards is sealed with that device's own secret.

`SealedBox` (`sealed_box.dart`) is AES-256-GCM. A message is JSON `{n: nonce, c: ciphertext, t: tag}`, all base64.

17.4 Joining and merging

`join(urls, code, …)`: after pairing, the member puts **all** its rows in the outbox (`enqueueEverything`) so the two shops merge, except shop settings, where the hub's win. A fresh install joining from the first screen (`fresh: true`) first clears its starting defaults, so they do not override or duplicate the hub's.

17.5 The sync loop (member)

```
loop:
  batch   = RowStore.pending(limit 300)          latest state of each changed row
  request = seal({push: batch, since: cursor, own: resyncing?, v: appVersion}, secret)
  POST /sync  (header carries the device ID)
  hub: apply each pushed row (last write wins), log every version in hub_changes,
       reply seal({pull: up to 400 log rows after `since` not from this device,
                   cursor, more, rejected, epoch, license})
  member: in one transaction apply pulled rows, acknowledge the outbox,
          save the cursor and epoch; store the hub's licence
  repeat while more / outbox ≥ 300 / epoch changed
```

Conflict rule (`RowStore.apply`): an upsert that only replaces the row when `excluded.updated_at > current.updated_at` , or when they are equal and the incoming `device_id` is ≥ the current one. Every device therefore ends up with the same row. Columns this version does not know are dropped, so an older app can take rows from a newer one. Rows that break a local constraint are counted as `rejected` .

17.6 Recovery cases

- **Epoch.** After the hub's database is restored, its log restarts with a new epoch. A member that sees a different epoch resets its cursor and sends everything again.
- **Resync after restore** (`hub.resync` meta). A restored device re-sends everything, so edits made elsewhere since the backup win back. A member also pulls the whole log, its own changes included (`own: true`).
- **The hub moved** (new IP address): on a network failure the member listens for the beacon (`_findHub`).
- **Revoked device:** the hub answers `revoked` and the member drops the hub's licence.
- `duplicates()` lists names that appear twice after a merge (products, customers, staff), for tidying by hand.

Errors travel by name (`HubError`) and are shown in the device's language.

17.7 Tests

`test/services/hub_service_test.dart` runs a real hub and member against two in-memory databases on `127.0.0.1` with `port: 0` (an ephemeral port), `beaconPort: 0` and a one-hour tick, then calls `syncNow()` by hand. Copy that setup for new hub tests.

18. Branches

`MDV-02`: a shop can have several locations.

- **A branch is the devices placed in it.** `device_branches` maps device ID → branch ID. Devices not listed are in the main branch (`mainBranchId = 'main'`).
- **Rows are not tagged with a branch.** A sale or stock movement belongs to the branch of the device that wrote it (`deviceId`). Old data needed no migration. The catch: moving a device to another branch moves its history too, and the app warns before doing it.
- `branch_scope.dart`: `db.branchFilter(branchId)` returns a `BranchFilter` usable in raw SQL (`filter.sql('s.device_id')` with `filter.variables`) or in the query builder (`filter.on(table.deviceId)`). The main branch is "every device except those placed elsewhere".
- **Stock is counted per branch.** Reports and the dashboard show all branches or one (`BranchService.viewing`). Branch choices are hidden until a second open branch exists.
- **Transfers** (`TransferRepository`): stock leaves immediately (`transferOut`), is in transit, and arrives as `transferIn` when the other branch counts it. Differences are flagged.
- **Branch data file** (`BranchMerge`, for branches with no network between them): an encrypted backup from the other branch is **merged** row by row using the same last-write-wins rule, after a safety backup. Importing the same file twice changes nothing, and what came in is queued for the local hub.
- Not built: per-branch prices, per-branch P&L, branch details on receipts.

19. Backups, restore and training mode

19.1 Backups (`backup_service.dart`)

Kind	How
Local snapshot	a consistent copy of the open database into `support/backups/aby—<timestamp>.sqlite`, checked after copying; the newest `backup.keep` (14) are kept. Made daily at start-up (`autoBackupIfDue`), at day close, by *Back up now*, and before every restore or merge.
Off-device	`.abybak`: a fresh snapshot encrypted with the backup password, shared (WhatsApp, Drive, email) or saved to a folder or USB stick. The status centre turns amber after `backup.offsiteReminderDays` (7) without one.

The `.abybak` format (`backup_codec.dart`):

```
"ABYBAK1\n" | header length (uint32 BE) | header JSON | ciphertext | GCM tag (16 bytes)
header: format, version, createdAt, kdf {pbkdf2-hmac-sha256, 120 000 iterations, salt}, cipher {aes-256-gcm, nonce}
```

The header is the GCM associated data, so changing any byte makes decryption fail. Key derivation runs in an isolate. Only a hash of the backup password is kept (`meta.backup_password_hash`), and the password is at least 6 characters.

19.2 Restore

`inspectFile` decrypts to a temp copy and checks it in an isolate (a healthy Aby database; shop name, sales count, whether it came from this device and whether it was hub-linked). `restore` then:

1. refuses in training mode;
2. **stages** the backup as `aby_pos.sqlite.restoring` next to the live file;
3. edits the staged copy in an isolate: leaves a pending-restore note, removes training markers, **keeps this device's identity** (device ID, hub link, paired devices and their revocations), unless the user chose to *replace* the device the backup came from, and marks it for resync;
4. makes a safety backup of the current data;
5. closes the database, deletes `-wal` / `-shm` / `-journal`, and **renames** the staged file into place, so a failed copy never leaves half a database;
6. the caller restarts the app (`AppRestarter`). On the next start, `finishPendingRestore` writes the audit entry into the restored database.

19.3 Training mode (`training_service.dart`)

- `enter(data: sample | copy)` copies the real database to `aby_pos_training.sqlite`, marks it (`meta`), empties its outbox, creates the marker file, and restarts. With `sample`, `TrainingSeeder` replaces the catalogue and history with the shop kind's sample shop and plays four weeks of trading **through the real repositories** under `withClock`, so every ledger adds up.
- While training: the yellow band shows; the hub never starts; backups, restore and the monthly accountant pack are refused; the licence gate is open.
- `leave()` deletes the marker and restarts on the real data, untouched. `reset()` rebuilds the copy the same way it was made.

20. Hardware: printers, scales, scanners, screens

All hardware is per device (device-scoped settings) and optional. Printing never blocks a sale.

20.1 Printing (`services/printing/`)

- `PrintService` keeps a **queue** of `PrintJob`s and retries on a timer when the printer is off or out of paper. It also handles reprinting the last receipt (audited, because a second copy of a paid receipt is how a sale gets rung up twice), the drawer, and kitchen tickets (on `device.kitchenPrinter`, or the receipt printer).
- `PrinterLink` implementations:
  - **Network:** raw bytes to TCP port 9100.
  - **Windows (USB):** `WindowsSpoolerLink` sends a RAW job through the Windows spooler using FFI (`winspool.drv`), in an isolate.
  - **Bluetooth (Android):** `BluetoothLink` uses the `aby/bluetooth` method channel in `MainActivity.kt` (`bonded`, `write`, `read`) over the serial port profile.
- Printer choice is `AppSettings.receiptPrinter` (a `PrinterTarget` as JSON). Paper is 58 or 80 mm.

20.2 Scales (`services/scale/`)

- Windows: `SerialScaleLink` on a COM port through `kernel32` FFI (`BuildCommDCB`, 8N1).
- Android: `BluetoothScaleLink` over the same `aby/bluetooth` channel.
- `ScaleService.weigh()` returns grams, or throws `ScaleException` (unreachable, unreadable or still moving).
- Scale-printed **barcodes** (EAN-13 with a shop prefix) are parsed by `ScaleBarcode`, configured with `selling.scalePrefix` and `selling.scaleValue` (weight or price).

20.3 Camera scanning

`ui/barcode_scanner.dart` uses `mobile_scanner` (Android) for product barcodes, receipt QR codes, staff badges and hub pairing codes. Hardware barcode scanners work as keyboards in the till's search box.

20.4 Customer display, prep screen, "Now serving"

`DisplayService` serves three web pages from the till to any browser on the shop network (port `device.displayPort`, default 8080):

```
http://<till>:8080/c/<token>   customer display (cart, amount due, MoMo number + QR, thank you)
http://<till>:8080/k/<token>   prep / kitchen screen
http://<till>:8080/s/<token>   "Now serving" board
```

The secret token is in the links, so only people given a link can open them. Generating a new token invalidates old links. The pages are in `display_pages.dart`. Checkout drives the display's state (`paying`, `done`, `cancelPaying`).

20.5 Native code

Platform	Channel / file	Purpose
Android	`aby/bluetooth` in `MainActivity.kt`	Bluetooth printers and scales
macOS	`aby/folder_access` in `MainFlutterWindow.swift`	security-scoped bookmarks so the sandbox lets the app write to a chosen folder after a restart

Platform	Channel / file	Purpose
Windows	Dart FFI only (`advapi32` , `winspool` , `kernel32`)	machine GUID, RAW printing, COM ports

21. Background services

Service	What it does	When
`SessionService`	auto-lock	every 15 s
`HubService`	hub or member sync	every 3 s
`PriceScheduler`	puts scheduled prices into effect	at start, then hourly
`AccountsExportService`	last month's accountant pack to a folder, once per month, catching up to 12 missed months	at start, then hourly, if `device.accountsMonthly`
`BackupService.autoBackupIfDue`	daily local snapshot	at start
`NotificationService` + `AlertService`	low stock, overdue credit, backups due, day left open, old parked orders, expenses due → phone or Windows notifications, each kind at most once a day unless it grows; per-person opt-out	started after sign-in data exists
`InsightService`	dashboard insights (stock running out, falling margins, late customers, cash short), dismissible	on demand
`ClockService`	whether the signed-in person is clocked in (`staff.clockInToSell`)	reactive
`StatusService`	the status centre: local and off-device backups, audit chain, sync, printer, accountant pack	at start and after relevant actions
`PrintService`	print queue retries	timer while jobs wait

Background work must never block the till and must surface failures in the status centre rather than throwing to the UI.

22. Reports, documents and exports

- `ReportRepository` covers sales, times, products, payments, staff, customers, stock and tax for any `DateRange`, optionally scoped to a branch. `ProfitRepository` gives a simple profit and loss against the period before.
- **Documents** use one table model, `TableDoc` / `DocSheet` (`table_doc.dart`), which renders three ways:
  - **CSV** (`core/csv.dart`);
  - **PDF**, branded through `PdfBrand` (`pdf_brand.dart`): Figtree and JetBrains Mono, a midnight title band, lime accents, figure tiles, zebra tables, bar charts, and the `AbyExtra` font as a fallback for `₵` and Ghanaian letters (ɛ, ɔ, ŋ). `PdfBrand.plain` uses built-in fonts for tests;
  - **XLSX**, written by hand (`xlsx.dart`): shared strings, styles, a frozen header with filters, `SUBTOTAL` totals that respect filters, data bars and A4 print setup.
- **Accountant pack** (`accountant_pack.dart`): one zip with CSVs (sales, payments, expenses, purchases, tax, stock valuation), the same tables as `accounts.xlsx`, and a PDF summary, named `aby_accounts_<first>_<last>.zip`. **Keep the CSV headers stable**, because accountants' imports read them.
- `ui/export_dialog.dart` is the shared "export as CSV, Excel or PDF, then share or save" dialog.

23. User interface: design system, layout and theme

23.1 Shared widgets (`lib/app/ui/`)

Build screens from these rather than from scratch:

Widget / helper	Use
`SectionPage`	the frame of every shell section: title row with actions, then content
`GroupCard`, `InfoTile`, `EmptyState`	settings and detail layouts
`TileGrid`, `CompactButtons`	responsive card grids; compact buttons on phones
`confirmDialog`, `textInputDialog`, `choiceDialog`, `showProgress` / `hideProgress`	dialogs
`DialogButtons`	dialog actions laid out in a row, or in pairs on phones
`showMessage(text, tone:)` / `hideMessage()`	toasts: across the top on phones, top right on wide screens
`goBack(result:)`	close the top dialog or page; **use this, not `Get.back`** (§29)
`PinPad`	PIN entry (standard, shuffled or keyboard, per `device.pinPad`)
`showApprovalPrompt`	manager-override prompt (wired into `PermissionService`)
`PeriodBar`, `BranchPicker`, `CustomerPicker`, `PhotoPicker`, `OptionChip`, `StatusDot`, `StaffAvatar`, `ProductPicture`, `charts.dart`	common pieces
`format.dart`	`relativeTime`, `formatDateTime`, `formatDate`
`*_labels.dart`, `*_text.dart`	readable names for permissions, settings, alerts, insights, loyalty

23.2 Layout

`ui/layout.dart`:

```
AbyBreakpoints.wide   = 840   // rail navigation, side-by-side list and detail
AbyBreakpoints.narrow = 600   // phones in portrait
context.isWide / context.isNarrow / context.screenWidth
```

On tablets and desktops, list screens show the list on the left and the detail on the right; on phones the detail is a pushed page. `test/widget/responsive_test.dart` covers phone widths. Check new screens at 360 px wide.

23.3 Theme and brand

`lib/app/theme/` is copied as-is from `aby-brand-kit/flutter/lib/theme`. Do not edit it in place; change the brand kit and copy it over. Its contents:

- **Colours** (`AbyColors` , a `ThemeExtension` with light and dark variants): midnight `#101322` , lime `#C8F560` , chalk `#F6F7F2` , blue `#3446E0` , coral `#FF6B4A` , and others.
- **Spacing** `AbySpace` (4, 8, 12, 16, 24, 32), **radii** `AbyRadius` (8, 14, 24), **sizes** `AbySize` (48 px minimum tap target; 56 px for POS buttons and payment chips).
- **Type** `AbyText` : Figtree for words; JetBrains Mono for receipts; tabular figures for prices and totals.
- **Brand rule:** lime is only for the Pay button, the waiting dot and cart counts.
- The theme follows the system unless `device.theme` is set.
- Full guidance is in `Aby-brand-guide.md` and `aby-brand-kit/docs/` (`screen-patterns.md` , `messages.md`).

24. Localisation

- All new UI text goes in `lib/l10n/app_en.arb` (about 1,950 keys). Generated classes are `AppLocalizations` (`nullable-getter: false`).
- In widgets: `context.l10n.someKey`. In controllers and services: `appL10n().someKey` (`ui/l10n.dart`).
- Placeholders use ARB metadata (`"supplierPaid": "Paid {amount}"` with an `@supplierPaid` entry).
- Some older selling screens still have English literals; move them over when you rework them. Repository exception messages are English for now.
- Printed receipts, PDFs and accountant files are deliberately English.
- Only `en` ships today (`device.language` exists for later). Copy follows the brand voice: plain words, no jargon, no emoji.

25. Testing

25.1 Layout

```
test/
├── helpers.dart            memoryDb(), FakeDevice, Repos (the whole repository graph), fixtures
├── data/                   repository tests against in-memory SQLite (most of the suite)
├── domain/                 pure engines: pricing, promotions, settlement, receipts, documents
├── services/               backup, hub, licence, printing, training seeder, money, PINs, ...
├── modules/                controller tests (cart, checkout)
├── widget/                 widget tests: till flow, PIN pad, dialogs, responsive layouts
└── generated_migrations/   drift schema snapshots (generated, do not edit)
```

25.2 Helpers

```
final db = memoryDb();               // in-memory, seeded with the Scoops Tamale menu
final empty = memoryDb(seed: false); // a fresh install
final r = await Repos.open(db);      // every repository on one database
final till2 = Repos.onDevice(db, 'device-2');   // a second device writing to the same db

await r.sales.completeSale(
  lines: [...],
  payment: const PaymentInput.cash(tendered: 2000),   // GH₵ 20.00 handed over
  cashier: cashier,
);
```

`owner` and `cashier` are ready-made `Staff`. Catalogue fixtures (`vanillaCone`, `toppings`, `single`, …) have prices in pesewas. `memoryDb` uses `closeStreamsSynchronously: true`, so widget tests are not left with drift's cleanup timer.

25.3 Guidelines

- **Test through repositories against a real in-memory database**, not mocks. That is how the existing suite works, and it catches SQL and transaction mistakes.
- **Fix time** with `withClock(Clock.fixed(...), () async { ... })` from `package:clock` for anything about business days, schedules or expiry.
- **Services take seams** for files, clocks and timers (`BackupService(backupsDirectory:, tempDirectory:, liveDatabaseFile:, iterations:)`, `LicenseService(machineId:, file:, publicKey:)`, `HubService(port: 0, every: …)`, `SessionService(clock:)`). Use them, and add them to new services.
- Use lower PBKDF2 `iterations` in tests to keep them fast.
- **Widget tests** build `AbyApp(openDatabase: () ⇒ memoryDb(), requireLicense: false)`.
- Every migration gets a case in `migration_test.dart` (§10).
- Before committing: `flutter analyze` with zero issues and `flutter test` all green.

26. Building and releasing

26.1 Version

`pubspec.yaml` has `version: 1.0.0+1`. The Windows script reads `major.minor.patch` from it. **Also update `appVersion`** in `app_version.dart`, which Settings → About shows and members report to the hub. `test/app_version_test.dart` and `build_windows.ps1` both fail when the two differ.

26.2 Windows installer

```
powershell -ExecutionPolicy Bypass -File build_windows.ps1
```

`build_windows.ps1`:

1. a clean `flutter build windows --release --obfuscate --split-debug-info=build\symbols\<version>`;
2. copies `msvcp140.dll`, `vcruntime140.dll` and `vcruntime140_1.dll` from `System32` next to `aby_pos.exe`, so Aby starts on a fresh Windows without the VC++ runtime;
3. runs Inno Setup on `installer/aby_pos.iss` → `build\installer\AbySetup—<version>.exe`;
4. when `$env:ABY_SIGN_CERT` holds the SHA-1 thumbprint of a code-signing certificate in the Windows certificate store, signs `aby_pos.exe`, the installer and the uninstaller with `signtool` (Windows SDK), timestamped. Without it the installer is unsigned, the script warns, and Windows SmartScreen warns shops when they run it.

Keep `build\symbols\<version>` for every shipped version (it is needed to read obfuscated crash stacks), and never ship it.

The installer:

- installs per machine to Program Files (needs admin) on 64-bit Windows 10 or later;
- shows the EULA (`installer/eula.txt`) with publisher *Abytrone Technologies*;
- closes a running Aby before upgrading;
- adds (and on uninstall removes) a Windows Firewall rule for the hub, on every network profile (shop Wi-Fi is often marked Public) but only from the local subnet;
- never touches shop data.

26.3 Crash log

`CrashLog.install()` (`crash_log.dart`), called in `main`, catches what nothing else did: `FlutterError.onError` (still shown in debug) and `PlatformDispatcher.onError`. The hub logs its own request failures there too, and replies only `{"error": "failed"}`. Each entry has the time, version, platform, error and stack, appended to `logs/aby_pos.log` in the support folder; past 512 KB it becomes `aby_pos.old.log` and a new one starts. A shop sends it from Settings → About → **Error log**. Release stacks are obfuscated: read them with `flutter symbolize -i <file> -d build\symbols\<version>\app.windows-x64.symbols`.

Never change `AppId`: it is how Windows knows a new installer upgrades the old one. Code signing is prepared (`SignTool=` is commented out) for when there is a certificate.

26.3 Android

`flutter build apk --release` or `flutter build appbundle`. The application ID is `com.aby.aby_pos`. Permissions: `INTERNET`, `BLUETOOTH` / `BLUETOOTH_ADMIN` (API ≤ 30), `BLUETOOTH_CONNECT`, `CAMERA`; cleartext

HTTP is allowed for the hub and display server. Release signing is not configured in the repo; set it up in `android/app/build.gradle.kts` with a keystore kept outside git.

26.4 macOS

`flutter build macos`. The app is sandboxed, with entitlements for network client **and server** (hub, display), user-selected files (read/write) and app-scoped bookmarks (the accounts folder). Both `DebugProfile.entitlements` and `Release.entitlements` must carry new entitlements.

26.5 Icons

Put the PNGs from `aby-brand-kit/icons/` into `assets/icons/`, then run `dart run flutter_launcher_icons -f flutter_launcher_icons.yaml` (Android adaptive icon on midnight `#101322`, Windows 256 px).

27. Coding conventions

From `analysis_options.yaml`, the README and the code itself:

- **Lints:** `flutter_lints` plus `strict-casts`, `strict-raw-types`, `prefer_single_quotes`, `always_declare_return_types`, `avoid_dynamic_calls` and `unawaited_futures`. Fire-and-forget futures are written `unawaited(f())`.
- **Comments say why, in plain English,** and refer to feature IDs (`MDV-02`, `INT-05`). Doc comments (`///`) on public members describe behaviour and units ("Pesewas.", "Base units."). Match the existing density: a short doc comment on most public members, few inline comments.
- **Naming:** drift rows end in `Row`; plain models have no suffix; drafts for forms end in `Draft` (`models/drafts.dart`); exceptions end in `Exception`.
- **Small, explicit types** over maps. Records (`(String, int)`) for small internal tuples.
- **Formatting:** keep diffs focused. Do not reformat whole files you only touched in a few lines (see §29).
- **Plug-in points** for future work are marked `TODO(sync)` and `TODO(storage)`.
- **Commits** are per phase or feature, with a one-line summary and a body listing feature IDs and schema changes. `last_report.md` and the *Status* paragraphs in FEATURES.md record what each phase built, the decisions made and what was left out.

28. Recipes: common changes step by step

Add a setting

1. Declare it in `AppSettings` with key, scope and default, and add it to `AppSettings.all`.
2. Add its label in `ui/setting_labels.dart` (for the activity log) and the UI in the right Settings page.
3. Read it with `settings.get(...)` (inside `Obx` for live updates); write it with `settings.set(..., by:)` after `authorize(Permission.changeSettings, ...)`.

Add a permission

1. Add a value to `Permission` in the right group, at the end of that group. Never rename existing ones.
2. Decide the built-in defaults in `BuiltInRole.defaultPermissions`. Existing databases need a migration step to grant it (see `from18To19` / `_grantToManager`).
3. Add its label in `ui/permission_labels.dart` and ARB text.
4. Use it through `PermissionService.authorize`. If it should show a shell section, add it to that section's `anyOf`.
5. Decide whether it works unlicensed (group `reports` or `admin`).

Add a column or table

1. Follow §10.1 exactly. Syncable tables mix in `SyncColumns`, and new tables go in the `@DriftDatabase` list.
2. Write through `insertTracked` / `updateTracked` inside a transaction.
3. The hub picks up any table with `id`, `updated_at`, `deleted_at` and `device_id` automatically (`RowStore.synced`). Nothing else is needed to sync it.
4. Add a model in `models/`, mapping in the repository, and tests.

Add a screen with its own route

1. `modules/foo/{bindings,controllers,views,widgets}`.
2. Add `Routes.foo` (document its argument) and a `GetPage` with `AuthMiddleware` in `AppPages`.
3. Navigate with `Get.toNamed<void>(Routes.foo, arguments: id)`.

Add a shell section

1. Add a value to `ShellSection` with icons, workspace and `anyOf` permissions, and a label.
2. Render a `SectionPage` from the shell view's switch.

Add an audited action

1. Add an `AuditAction` value at the end.
2. In the repository, inside the action's transaction: `await _audit.record(action: AuditAction.foo, actor: by, summary: '…', entity: 'foo', entityId: id, details: {'before': …, 'after': …});`
3. Give it a readable rendering in `domain/audit_details.dart` if it has details.

Add UI text

1. Add the key (and `@key` metadata for placeholders) to `app_en.arb`.
2. Run `flutter gen-l10n` (or any `flutter run` or `flutter test`).
3. Use `context.l10n.key` or `appL10n().key`.

Add a hardware link

Implement `PrinterLink` or `ScaleLink`, choose it from the device setting's JSON target, and keep platform calls behind the interface so tests can use fakes.

29. Pitfalls and troubleshooting

Symptom	Cause and fix
A dialog will not close; a toast disappears instead	`Get.back()` closes a GetX snackbar first. Use `goBack()` from `ui/dialogs.dart`.
`app_database.g.dart` is not regenerated, with no clear error	Code placed between the `@DriftDatabase(...)` annotation and `class AppDatabase` (this happened with `mainBranchId`). Keep declarations **above** the annotation.
"Symlink support required" on Windows	Turn on Developer Mode.
A change syncs but loses to an older value	`updatedAt` was not set to `clock.now()` on the update.
Duplicate built-in rows after sync	The row was created with `newId()` instead of a stable ID (`settingRowId`, `reasonRowId`, a fixed `BuiltInRole` ID).
Dates are wrong in raw SQL or fixtures	Dates are Unix **seconds** (`build.yaml`); `audit_log.at_ms` is milliseconds.
Times are wrong in tests or the training seeder	Use `clock.now()`, not `DateTime.now()`, in repositories.
A service points at a closed database after restore	It was cached statically. Always `Get.find` (or inject) after `AppRestarter`.
The till cannot sell during development	No licence. Use training mode, or a dev licence (§2, §16).
Members cannot find the hub	Routers often block UDP broadcast: add the hub by address and code instead. Check the Windows firewall rule (the installer adds it; `flutter run` builds do not).
A macOS export folder stops working after a restart	The sandbox needs the security-scoped bookmark (`FolderAccess`); store a new one when the folder is chosen.
An enum value was renamed and old rows fail to load	Enums are stored by name (`textEnum`, permissions, audit actions, settings strings). Add a migration or keep the old name.
`dart format` rewrites a whole file	Don't commit mass reformatting; keep the diff to the lines you changed.
The migration test fails after a schema change	You missed a step in §10.1, often step 6 (`schema generate`) or the loop bound.
Approval prompt never appears in a test	`PermissionService.prompter` is only set in `InitialBinding`; set it yourself or sign in as someone with the permission.

30. What is not built yet

From FEATURES.md Phase 8 and the `TODO(sync)` markers:

- **Aby Cloud** (`INT-01`): cloud sync and an owner portal against a Laravel API. The outbox already holds JSON payloads; the wire format (dates as epoch milliseconds, enums by name) is to be agreed. `syncedAt` and `OutboxRepository.acknowledge` are the plug-in points.
- Photo upload as files (product, staff and receipt photos stay on the device).
- PIN hash handling on the server (`StaffRepository` TODO).
- MoMo payment APIs (`INT-02` ; MoMo checks are manual today), SMS gateway (`INT-03`), GRA E-VAT (`INT-04`), an accounting API (`INT-05` , beyond the monthly pack), online ordering (`INT-06`).
- Per-branch prices and P&L; branch details on receipts.
- More languages.

Open product questions are listed in `FEATURES.md §20`.

31. Glossary

Term	Meaning
Pesewa	1/100 of a Ghana cedi (GH₵). All amounts are integer pesewas.
Business day	trading day starting at 05:00 by default
Shift	a drawer session opened with a counted float; **X report** = shift so far
Close day / Z report	end-of-day cash-up that locks the day's sales against voids
No-sale	opening the drawer without a sale (needs a reason)
Pay-in / pay-out	cash put into or taken out of the drawer
Safe drop	cash moved from the drawer to the safe
MoMo	mobile money (MTN MoMo, Telecel Cash, AT Money)
Credit book	customers buying on account; balances in `customer_ledger`
Wallet	money held for a customer (paid ahead, kept change, refunds)
Parked order	a cart put aside under a name; also how the live cart is saved
Variant	a size or price point of a product (every product has at least one)
Option group	modifiers such as Toppings, with min, max and repeats
Stock item	what is counted (pieces, g, ml); products use stock items through **recipes**
Movement	one entry in the stock ledger
Outbox	rows changed on this device and not yet sent
Hub / member	the device serving the shop's Wi-Fi sync, and the devices syncing with it
Epoch	the hub change log's generation; changes after a hub restore
Branch	a shop location; a set of devices
Training mode	practice on a copy of the data, never mixed with the real books
Device code	the 16-character code a licence is issued for
Accountant pack	month-end zip of CSV, XLSX and PDF for the accountant